Final Project:

Autonomous Service Robot in a Simulated Town Using ROS 2 and Nav2

https://github.com/MSFEALabsRobotics/AMR-S2026/blob/main/Final%20Project.md

1. Project Overview

The aim of this project is to develop an autonomous mobile service robot capable of exploring a simulated town, building a map of the environment, identifying important landmarks, and later executing service missions fully programmatically using the ROS 2 and Nav2 framework.

The project is divided into two main parts. In the first part, the robot must autonomously explore an unknown town, generate a map using SLAM, and detect a set of predefined landmarks represented by QR codes. In the second part, the robot must execute user-requested missions through a custom Python graphical user interface, without relying on RViz mouse clicks for navigation commands. All navigation goals, waypoint selection, and mission execution must be handled in software.

The project combines concepts of localization, mapping, SLAM, landmark-based environment understanding, autonomous mission planning, and navigation in dynamic environments.

2. Environment Description

The robot operates in a simulated town environment. The town contains roads, buildings, and a set of important service locations. These locations are represented by visible QR codes that serve as landmarks. The robot must be able to detect these landmarks during exploration and store their positions for later use in missions.

The town includes the following landmark locations:

- Pharmacy
- Firefighting center
- Supermarket
- Restaurant
- House 1
- House 2
- House 3
- House 4
- House 5
- Docking station for charging and waiting

3. Project Part 1: Autonomous Exploration and Mapping

In Part 1, the robot must explore the town autonomously and create a usable map of the environment.

Required tasks
The robot must:

- Explore the simulated town
- Build a map using a SLAM-based approach
- Detect the QR-code landmarks distributed in the town
- Associate each detected QR code with its landmark identity
- Estimate and store the location of each landmark in a structured form
- Save the final map for later navigation use

At the end of Part 1, the system should have:

- A saved 2D map of the town
- A list or database of landmark names and positions
- A method to reuse this information during mission execution

5. Project Part 2: Autonomous Mission Execution

In Part 2, the robot uses the map and landmark information obtained in Part 1 to perform service missions. The robot receives missions through a **custom Python GUI**. The user should only specify the type of mission and the target home. The rest of the process must be fully autonomous. After completing each mission, the robot must return to the docking station for charging or waiting.

Mission types
The robot must support the following missions:

A. Grocery delivery
The robot goes to the **supermarket** and then to a selected house.
Example: **Delivery to House 3** Sequence: Docking station -> Supermarket -> House 3 -> Docking station

B. Food delivery
The robot goes to the **restaurant** and then to a selected house.
Example: **Restaurant delivery to House 2** Sequence: Docking station -> Restaurant -> House 2 -> Docking station

C. Fire emergency
The robot goes to the **firefighting center** to pick up tools, then travels to a selected house.
Example: **Fire at House 5** Sequence: Docking station -> Firefighting center -> House 5 -> Docking station

D. Medical help
The robot goes to the **pharmacy** and then to a selected house.
Example: **Medical help to House 1** Sequence: Docking station -> Pharmacy -> House 1 -> Docking station

6. Dynamic Environment Requirement

During Part 2, the town must become a dynamic environment.

In addition to the main service robot, two more robots must move randomly in the town. These robots simulate traffic or moving obstacles. The main robot must navigate safely while avoiding them during mission execution.

7. Human-Robot Interface

The user must not control the robot through RViz mouse clicks or manually selected goals on the map. Instead, a small **custom Python GUI** must be developed. The GUI serves as the mission input interface.

Example inputs

- Grocery delivery to Home 1
- Restaurant delivery to Home 4
- Fire at Home 2
- Medical help to Home 5

The GUI should allow the user to:

- Select mission type
- Select target home
- Send mission request to the robot

Once the mission is submitted, the software must automatically:

- Look up the required landmark locations
- Build the mission sequence
- Send navigation goals programmatically
- Monitor mission completion
- Return the robot to the docking station

8. Sensor Constraints

The robot must operate under the following sensor limitations:

- **2D LiDAR** must be the main sensor used for mapping, localization, obstacle detection, and navigation.
- **IMU** may be used optionally to improve heading estimation or motion stability.
- **Camera** may be used only for detecting and identifying landmarks represented by QR codes.
- The **camera must not be used for navigation, localization, obstacle avoidance, or path planning**.
- **GPS is not allowed** in this project setup.

Accordingly, the navigation and mapping framework must rely primarily on LiDAR-based SLAM and Nav2-compatible localization, while the camera is reserved strictly for landmark recognition.

9. Functional Requirements

The final system must satisfy the following requirements:

Exploration and mapping

- Autonomous exploration of the town
- Map creation using SLAM
- Detection and identification of QR-code landmarks
- Storage of landmark positions

Autonomous mission logic

- Programmatic mission planning

- No manual goal selection in RViz
- Automatic waypoint generation from stored landmark data
- Correct execution of multi-step missions
- Automatic return to docking station after every mission

Dynamic navigation

- Presence of two additional moving robots
- Collision avoidance with moving traffic robots
- Safe motion in a dynamic environment

User interface

- Custom Python GUI
- Simple mission input
- Clear command structure for task selection

10. Suggested System Architecture

A possible implementation may include the following modules:

- **SLAM and map generation node**
- **QR-code detection node**
- **Landmark database or configuration file**
- **Mission planner node**
- **Nav2 interface node**
- **Python GUI**
- **Traffic robot controllers**
- **Docking/return logic**

The mission planner is responsible for converting a user request into an ordered list of destination waypoints. The robot should then navigate through these waypoints using ROS 2 and Nav2.

11. Expected Deliverables

Each project team should submit the following:

1. **Simulation environment** of the town
2. **Working robot model** integrated with ROS 2 and Nav2
3. **SLAM-based map** of the town
4. **Landmark detection system** using QR codes
5. **Stored landmark location data**
6. **Custom Python GUI** for mission input
7. **Autonomous mission execution software**
8. **Dynamic obstacle/traffic simulation** using two additional robots
9. **Final report** explaining design, implementation, and testing
10. **Demonstration video** showing exploration and mission execution

12. Minimum Demonstration Scenario

At minimum, the final demonstration should show:

- The robot exploring the town
- Generation and saving of a map

- Detection of all required landmarks
- Execution of at least one mission from the GUI
- Avoidance of moving robots during the mission
- Return to docking station after completing the task

13. Recommended Evaluation Criteria

The grading breakdown is as follows:

- Environment design and simulation quality: **10%**
- SLAM and map generation: **20%**
- QR-code landmark detection and storage: **15%**
- Mission planning and autonomous execution: **25%**
- Dynamic obstacle avoidance: **15%**
- Python GUI and system integration: **10%**
- Report and presentation quality: **5%**

14. Project Constraints

The following constraints apply to the project:

- The robot must be commanded **programmatically**.
- **RViz mouse clicks** must not be used for mission execution.
- The mission must be selected only by specifying task type and destination through the custom GUI.
- Landmark positions must be obtained from the robot's mapping and exploration process.
- The robot must return to the docking station after each mission.
- Only the following sensors are allowed:
    - **2D LiDAR** for mapping, localization, and navigation
    - **IMU (optional)** for improved orientation estimation
    - **Camera** for landmark/QR-code detection only
- The **camera may not be used for navigation**.
- **GPS is not allowed** in this setup.

15. Concluding Statement

This project aims to simulate a realistic autonomous service-robot application in a town-like environment. It requires the integration of SLAM, landmark recognition, autonomous navigation, mission planning, and dynamic obstacle avoidance within ROS 2 and Nav2. The result should be a complete software system in which a user can request a mission at a high level, and the robot handles the rest autonomously.